



## MILESTONE 2

# Ingestion to Bronze

A single metadata-driven Data Factory pipeline that lands every source in the Bronze lake



STEP 1

## Create ADLS Gen2 Storage

### WHAT

Created a Standard, locally-redundant (LRS) storage account with Hierarchical namespace enabled, in rg-atliq-capstone, region Central India, then added a container named lakehouse.

### WHY

*Hierarchical namespace is what turns blob storage into a true data lake (real folders and abfss paths) that Databricks and the Medallion layers depend on. LRS keeps cost near zero for this data volume.*

# Create ADLS Gen2 Storage

The screenshot shows the Microsoft Azure portal interface for the 'atliqlakeab' Storage account. The left sidebar contains a navigation menu with options like Activity log, Tags, Diagnose and solve problems, Access Control (IAM), Data migration, Events, Storage browser, Partner solutions, Resource visualizer, Data storage, Containers, Classic file shares, Queues, Tables, Security + networking, Data management, Storage Actions, Redundancy, Data protection, Blob inventory, Static website, Lifecycle management, Settings, Configuration, Resource sharing (CORS), SFTP, and Advisor.

The main content area displays the 'Overview' page for the 'atliqlakeab' Storage account. It includes a search bar, a list of actions (Upload, Open in Explorer, Delete, Move, Refresh, Open in mobile, CLI / PS, Feedback), and a 'JSON View' link. The 'Essentials' section shows key account details:

| Property        | Value                                |
|-----------------|--------------------------------------|
| Resource group  | rg-atliq-capstone                    |
| Location        | centralindia                         |
| Subscription    | Azure subscription 1                 |
| Subscription ID | fe7a04e5-37b1-4992-a127-541400ea45ad |
| Disk state      | Available                            |
| Tags            | Add tags                             |

The 'Properties' tab is selected, showing settings for Data Lake Storage, File service, Queue service, and Table service. The 'Data Lake Storage' section includes settings like Hierarchical namespace (Enabled), Default access tier (Hot), Blob anonymous access (Disabled), Blob soft delete (Enabled (7 days)), Container soft delete (Enabled (7 days)), Versioning (Disabled), Change feed (Disabled), NFS v3 (Disabled), SFTP (Disabled), and Storage tasks assignments (None). The 'File service' section includes Identity-based access (Not configured), Default share-level permissions (Disabled), Soft delete (Enabled (7 days)), and Require Encryption in Transit for SMB (Security) (Enabled). The 'Queue service' section shows CMK support (Disabled). The 'Table service' section is currently empty.

The 'Security' section shows settings for Require secure transfer for REST API operations (Enabled), Storage account key access (Enabled), Minimum TLS version (Version 1.2), and Infrastructure encryption (Disabled). The 'Networking' section shows Public network access (Enabled), Public network access scope (Enable from all networks), Private endpoint connections (0), Network routing (Microsoft network routing), and Endpoint type (Standard).

atliqlakeab Overview: Hierarchical namespace Enabled, locally-redundant storage, Central India



STEP 2

## Land the Flat Files

### WHAT

Uploaded the two source CSVs, `supplier_price_list.csv` and `marketing_spend.csv`, into a `landing/` folder inside the lakehouse container.

### WHY

*These flat files are a second source type alongside the SQL database. Landing them first lets the same pipeline pick them up and convert them to Parquet, so Bronze holds every source in one consistent format.*

## Land the Flat Files

Microsoft Azure portal (PWA) - lakehouse - Microsoft Azure

Home > attliqlakeab\_1788003974775 | Overview > attliqlakeab | Containers

### lakehouse

Container

Search

+ Add Directory Upload Refresh Delete Copy Paste Rename Acquire lease Break lease Edit columns

lakehouse > landing

Authentication method: Access key ([Switch to Microsoft Entra user account](#))

Search blobs by prefix (case-sensitive)

Only show active objects

Showing all 2 items

| <input type="checkbox"/> | Name                                    | Last modified         | Access tier    | Blob type  | Size      | Lease state |
|--------------------------|-----------------------------------------|-----------------------|----------------|------------|-----------|-------------|
| <input type="checkbox"/> | [...]                                   |                       |                |            |           | ...         |
| <input type="checkbox"/> | <a href="#">marketing_spend.csv</a>     | 8/29/2026, 5:22:04 PM | Hot (Inferred) | Block blob | 22.07 KiB | Available   |
| <input type="checkbox"/> | <a href="#">supplier_price_list.csv</a> | 8/29/2026, 5:22:04 PM | Hot (Inferred) | Block blob | 1.4 KiB   | Available   |

Add or remove favorites by pressing Ctrl+L+Shift+F

lakehouse/landing/ holding marketing\_spend.csv and supplier\_price\_list.csv



STEP 3

## Create Azure Data Factory

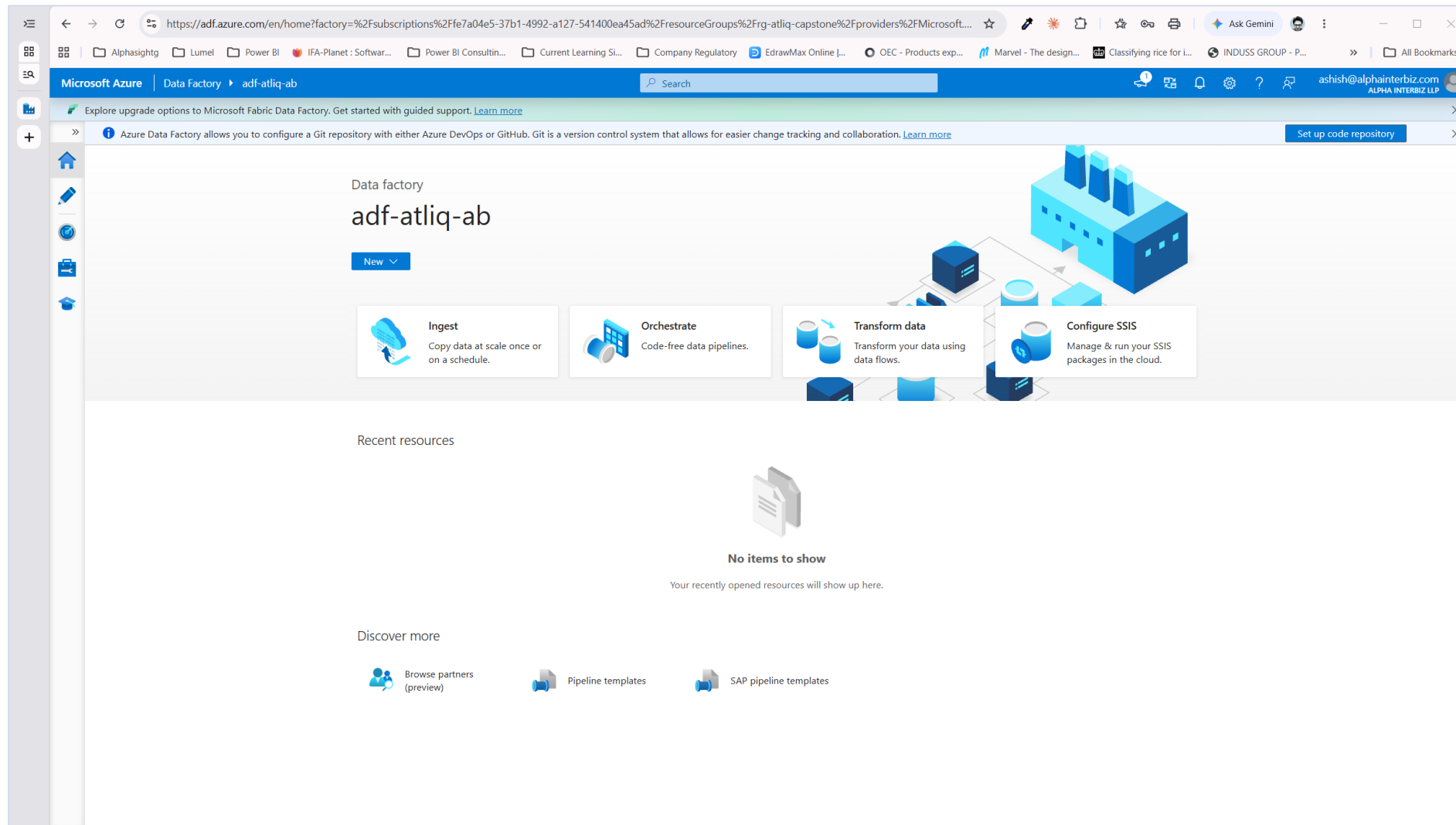
### WHAT

Created the Data Factory adf-atliq-ab in the same resource group and region, choosing Configure Git later so the first pipeline build stays simple.

### WHY

*Data Factory is the orchestration engine that reads the control table and moves data into Bronze. Git is deferred until the pipeline works, so version control is added at a clean, known-good checkpoint.*

# Create Azure Data Factory



ADF Studio home for adf-atliq-ab

# 4

STEP 4

## Secure the Password with Key Vault

### WHAT

Created Key Vault kv-atliq-ab (RBAC model), stored the SQL admin password as the secret sql-admin-password, and granted the Data Factory managed identity the Key Vault Secrets User role.

### WHY

*Secrets should never sit inside pipeline configuration. Centralizing the password in Key Vault, read at runtime via managed identity, is the security-hardening the reliability milestone rewards and matches real-world practice.*

# Secure the Password with Key Vault

The screenshot displays the Microsoft Azure portal interface. At the top, the navigation bar includes the 'Microsoft Azure' logo, a search bar, and user information for 'ashish@alphainterbiz.co...'. The main content area is titled 'kv-atliq-ab | Overview' and shows a green checkmark indicating 'Your deployment is complete'. Below this, deployment details are listed: 'Deployment name : kv-atliq-ab', 'Subscription : Azure subscription 1', and 'Resource group : rg-atliq-capstone'. The deployment occurred on '29/08/2026, 5:41:52 pm' with a 'Correlation ID : dda3eca2-ff33-46e1-b95d-b2672c9bac9f'. A 'Go to resource' button is provided. The left sidebar contains links for 'Overview', 'Inputs', 'Outputs', and 'Template'. The right sidebar features recommendations for 'Cost management', 'Microsoft Defender for Cloud', 'Free Microsoft tutorials', and 'Work with an expert'. A footer note at the bottom left reads 'Add or remove favorites by pressing Ctrl+Shift+F'.

Key Vault kv-atliq-ab deployment complete

## Secure the Password with Key Vault

Microsoft Azure portal (PWA) - kv-atliq-ab - Microsoft Azure

Home > kv-atliq-ab

### kv-atliq-ab | Secrets

Key vault

Search

+ Generate/Import Refresh Restore Backup Manage deleted secrets View sample code

The secret 'sql-admin-password' has been successfully created.

| Name               | Type | Status    | Expiration date |
|--------------------|------|-----------|-----------------|
| sql-admin-password |      | ✓ Enabled |                 |

Overview  
Activity log  
Access control (IAM)  
Tags  
Diagnose and solve problems  
Access policies  
Resource visualizer  
Events  
Objects  
Keys  
**Secrets**  
Certificates  
Settings  
Monitoring  
Automation  
Help

Add or remove favorites by pressing Ctrl+L+Shift+F+F

Give feedback

*kv-atliq-ab Secrets: sql-admin-password created and enabled*



STEP 5

## Create the Linked Services

### WHAT

Created three linked services: ls\_azuresql (SQL, password pulled from Key Vault), ls\_adls (the data lake), and ls\_keyvault (the vault itself). Each tested green before publishing.

### WHY

*Linked services are the connection definitions the pipeline reuses. Naming them by convention and testing each connection up front means the datasets and pipeline wire up cleanly with no credential surprises later.*

# Create the Linked Services

Microsoft Azure | Data Factory | adf-atliq-ab

Validate all | Publish all

Migrate to Fabric (Preview)

### Linked services

Linked service defines the connection information to a data store or compute. [Learn more](#)

+ New

Filter by name Annotations: Any

Showing 1 - 3 of 3 items

| Name        | Type                         | Related | Annotations |
|-------------|------------------------------|---------|-------------|
| ls_adls     | Azure Data Lake Storage Gen2 | 0       |             |
| ls_azuresql | Azure SQL Database           | 0       |             |
| ls_keyvault | Azure Key Vault              | 1       |             |

Linked services: ls\_adls, ls\_azuresql, ls\_keyvault



STEP 6

## Create the Datasets

### WHAT

Created five datasets: `ds_azuresql_control` (reads the control table), `ds_azuresql_generic` (parameterized on schema and table), `ds_adls_bronze_parquet` (parameterized on `folder_path` and `file_name`), and two CSV datasets.

### WHY

*Two datasets are parameterized so a single dataset serves every source. One generic SQL reader and one generic Bronze writer mean new tables need no new datasets, only a new control-table row. That is the metadata-driven design.*

## Create the Datasets

The screenshot displays the Microsoft Azure Data Factory portal interface. The left sidebar shows the 'Factory Resources' tree with 'Datasets' expanded, listing 'ds\_azuresql\_generic' and 'ds\_azuresql\_control'. The main workspace shows the configuration for 'ds\_azuresql\_generic', which is an 'Azure SQL Database' dataset. The 'Connection' tab is active, showing the 'Linked service' as 'ls\_azuresql' and the 'Table' as '@dataset().schema'. A 'Test connection' button indicates 'Connection successful'. The 'Properties' panel on the right shows the 'General' tab with the 'Name' field set to 'ds\_azuresql\_generic' and a 'Description' field. The 'Annotations' section has a '+ New' button. A 'Cancel' button is located at the bottom right of the configuration area.

*ds\_azuresql\_generic: dynamic schema and table name, connection successful*



STEP 7

## Capture the Run Timestamp

### WHAT

First activity, a Set variable that writes `@utcnow()` into a pipeline variable `run_start_at` at the very start of the run.

### WHY

*One timestamp captured at the start is reused for every table's watermark and Bronze folder. Anything that arrives mid-run is caught on the next run instead of being skipped, a no-data-loss guarantee.*

# Capture the Run Timestamp

The screenshot displays the Microsoft Azure Data Factory portal interface. The browser address bar shows the URL: `https://adf.azure.com/en/authoring/pipeline/pipeline1?factory=%2Fsubscriptions%2Ffe7a04e5-37b1-4992-a127-541400ea45ad%2FresourceGroups%2Frg-atliq-capstone%2F...`. The page title is "Microsoft Azure | Data Factory | adf-atliq-ab".

The left sidebar shows the "Factory Resources" section with a search bar and a list of resources:

- Pipelines: 1
  - pl\_ingest\_bronze
- Change Data Capture (preview): 0
- Datasets: 5
  - ds\_adls\_bronze\_parquet
  - ds\_adls\_csv\_marketing
  - ds\_adls\_csv\_supplier
  - ds\_azuresql\_control
  - ds\_azuresql\_generic
- Data flows: 0
- Power Query: 0

The main workspace shows the "Activities" section with a search bar and a list of activities:

- set
- (X) Set variable

The "Set variable" activity is selected, and its configuration is shown in the "Properties" panel. The "General" tab is active, showing the following details:

- Name: pl\_ingest\_bronze
- Description: (empty)
- Annotations: + New

The "Settings" tab is also visible, showing the following configuration:

- Variable type: ☒ Pipeline variable ☐ Pipeline return value
- Name: run\_start\_at
- Value: @utcnow()

A "Cancel" button is located at the bottom right of the "Properties" panel.

Set variable: `run_start_at = @utcnow()`



STEP 8

## Read the Control Table

### WHAT

A Lookup activity runs `SELECT ... FROM etl.control_table` with First row only disabled, returning all five source rows as an array.

### WHY

*This is the metadata-driven mechanism in action: the pipeline asks the control table what to load and how, rather than hard-coding tables. First row only must be off, or the loop would process only the first source.*

# Read the Control Table

The screenshot shows the Microsoft Azure Data Factory interface. The left sidebar displays the 'Factory Resources' tree with the following structure:

- Pipelines: 1
  - pl\_ingest\_bronze
- Change Data Capture (preview): 0
- Datasets: 5
  - ds\_adls\_bronze\_parquet
  - ds\_adls\_csv\_marketing
  - ds\_adls\_csv\_supplier
  - ds\_azureSQLControl
  - ds\_azureSQLgeneric
- Data flows: 0
- Power Query: 0

The main canvas shows the 'pl\_ingest\_bronze' pipeline configuration. The 'Settings' tab is active, displaying the following configuration:

- Source dataset: ds\_azureSQLControl
- First row only: ☐
- Use query: ☒ Table ☐ Query ☐ Stored procedure
- Query: 

```
SELECT table_name, source_schema, load_type, watermark_column, last_loaded_at FROM etl.control_table
```
- Query timeout (minutes): 120
- Isolation level: Select...
- Partition option: ☒ None ☐ Physical partitions of table ☐ D...

A 'Preview data' dialog is open, showing the following data:

|   | table_name  | source_schema | load_type   | watermark_column | last_loaded_at      |
|---|-------------|---------------|-------------|------------------|---------------------|
| 1 | customers   | dbo           | full        |                  | 1900-01-01T00:00:00 |
| 2 | order_items | dbo           | incremental | created_at       | 1900-01-01T00:00:00 |
| 3 | orders      | dbo           | incremental | updated_at       | 1900-01-01T00:00:00 |
| 4 | payments    | dbo           | incremental | updated_at       | 1900-01-01T00:00:00 |
| 5 | products    | dbo           | full        |                  | 1900-01-01T00:00:00 |

The 'Preview data' dialog also shows the linked service 'ls\_azureSQL' and the object 'etl.control\_table'. A 'Cancel' button is visible at the bottom right of the dialog.

Lookup on etl.control\_table with First row only off, previewing the 5 control rows



STEP 9

## Loop and Branch by Load Type

### WHAT

A ForEach iterates the control-table array (parallel, batch count 4). Inside it, an If Condition on `@equals(item().load_type, 'incremental')` splits full loads from incremental loads.

### WHY

*One loop plus one branch applies the correct strategy per source, decided by metadata. Parallel with batch count 4 loads independent tables concurrently while protecting the free-tier SQL database from too many connections.*

## Loop and Branch by Load Type

The screenshot displays the Microsoft Azure Data Factory (ADF) interface for a pipeline named 'pl\_ingest\_bronze'. The pipeline is composed of three activities: 'Set variable', 'Lookup', and 'ForEach'. The 'Set variable' activity is configured to set the variable 'run\_start\_at'. The 'Lookup' activity is configured to look up the 'control table'. The 'ForEach' activity is configured to iterate over the output of the 'Lookup' activity. The 'ForEach' activity is set to 'Parallel' execution, with a 'Batch count' of 4 and 'Items' set to '@activity('Lookup control\_table').out...'. The 'Properties' pane on the right shows the 'General' tab with the name 'pl\_ingest\_bronze' and a description field. The 'Settings' tab for the 'ForEach' activity is also visible, showing the configuration for parallel execution and batch count.

*ForEach settings: parallel, batch count 4, items from the Lookup output*

## Loop and Branch by Load Type

The screenshot displays the Microsoft Azure Data Factory portal interface. The top navigation bar shows the URL: [https://adf.azure.com/en/authoring/pipeline/pl\\_ingest\\_bronze?factory=%2Fsubscriptions%2Ffe7a04e5-37b1-4992-a127-541400ea45ad%2FresourceGroups%2Frg-atliq-capsto...](https://adf.azure.com/en/authoring/pipeline/pl_ingest_bronze?factory=%2Fsubscriptions%2Ffe7a04e5-37b1-4992-a127-541400ea45ad%2FresourceGroups%2Frg-atliq-capsto...). The left sidebar shows the 'Factory Resources' tree with 'Pipelines' expanded, listing 'pl\_ingest\_bronze'. The main canvas shows the pipeline 'pl\_ingest\_bronze' with a 'ForEach source' loop. An 'If Condition' activity is added to the loop, with the condition 'If incremental'. The 'Properties' pane on the right shows the 'General' tab with the name 'pl\_ingest\_bronze'. The 'Activities (3)' tab is selected, showing the 'Expression' '@equals(item().load\_type, 'incremental...'' and the 'Case' table below.

| Case  | Activity                             |
|-------|--------------------------------------|
| True  | Copy incremental + Advance watermark |
| False | Copy full                            |

*If incremental: load\_type expression, True runs Copy incremental + Advance watermark, False runs Copy full*

# 10

STEP 10

## Copy Full vs Copy Incremental

### WHAT

False branch: Copy full runs `SELECT *` into `bronze/<table>/data.parquet`. True branch: Copy incremental copies only rows where `watermark > last_loaded_at` into a date-partitioned `bronze/<table>/ingest_date=YYYY-MM-DD/` folder.

### WHY

*Reference tables reload in whole (simple, cheap, latest-only). Transactional tables move just the delta and keep a per-date audit trail. Filtering in the source query means less data crosses the wire.*

# Copy Full vs Copy Incremental

The screenshot displays the Microsoft Azure Data Factory (ADF) web interface. The top navigation bar shows the URL and various icons. The left sidebar contains the 'Factory Resources' pane with a tree view showing Pipelines, Change Data Capture (preview), and Datasets. The main canvas shows the 'pl\_ingest\_bronze' pipeline with a 'ForEach source' loop containing an 'If incremental' condition. Inside the 'True activities' branch, there is a 'Copy data' activity. The 'Copy data' activity is configured with 'Copy incremental' as the copy type and is connected to a 'Stored procedure' activity named 'Advance watermark'. The 'Properties' pane on the right shows the 'General' tab for the 'pl\_ingest\_bronze' pipeline.

**Copy data activity configuration (Source tab):**

- Source dataset: ds\_azureql\_generic
- Dataset properties:

| Name       | Value                 | Type   |
|------------|-----------------------|--------|
| schema     | @item().source_schema | string |
| table_name | @item().table_name    | string |
- Use query: ☒ Query
- Query: @concat('SELECT \* FROM [', item().so...
- Query timeout (minutes): 120
- Isolation level: Select...
- Partition option: ☒ None
- Additional columns: + New

**Properties pane (General tab):**

- Name: pl\_ingest\_bronze
- Description:
- Annotations: + New

True branch, source: Copy incremental reads via a dynamic query from the control row (watermark filter)

## Copy Full vs Copy Incremental

The screenshot displays the Microsoft Azure Data Factory portal interface. The left sidebar shows the 'Factory Resources' tree with 'Pipelines' expanded, listing 'pl\_ingest\_bronze'. The main canvas shows the pipeline 'pl\_ingest\_bronze' with a 'Copy data' activity selected. The 'Copy data' activity is configured with the following settings:

- Source dataset:** ds\_azuresql\_generic
- Dataset properties:**

| Name       | Value                 | Type   |
|------------|-----------------------|--------|
| schema     | @item().source_schema | string |
| table_name | @item().table_name    | string |
- Use query:** Query (selected)
- Query:** @concat('SELECT \* FROM [', item().so...
- Query timeout (minutes):** 120
- Isolation level:** Select...
- Partition option:** None (selected)
- Additional columns:** + New

The 'Properties' panel on the right shows the 'General' tab with the 'Name' field set to 'pl\_ingest\_bronze' and a 'Description' field. The 'Annotations' section has a '+ New' button. The 'Cancel' button is at the bottom right.

False branch, source: Copy full runs a `SELECT *` built from the control row

## Copy Full vs Copy Incremental

The screenshot displays the Microsoft Azure Data Factory portal interface. The main canvas shows a pipeline named 'pl\_ingest\_bronze' with a 'Copy data' activity configured to 'Copy incremental'. This activity is connected to a 'Stored procedure' activity named 'Advance watermark'. The 'Sink dataset' is 'ds\_adls\_bronze\_parquet'. The 'Dataset properties' table shows the following configuration:

| Name        | Value                                    | Type   |
|-------------|------------------------------------------|--------|
| folder_path | @concat('bronze/', item().table_name...) | string |
| file_name   | data.parquet                             | string |

The 'Copy behavior' is set to 'Select...'. The 'Max concurrent connections' is 1, 'Block size (MB)' is 1024, 'Metadata' is 'New', and 'Max rows per file' is 1000000. The 'Properties' panel on the right shows the 'Name' as 'pl\_ingest\_bronze' and 'Description' as an empty field.

True branch, sink: Bronze parquet with a dynamic folder path per table

## Copy Full vs Copy Incremental

The screenshot displays the Microsoft Azure Data Factory portal interface. The left sidebar shows the 'Factory Resources' tree with 'Pipelines' expanded, highlighting the 'pl\_ingest\_bronze' pipeline. The main canvas shows the 'Copy data' activity configuration for the 'pl\_ingest\_bronze' pipeline. The activity is currently set to 'Copy full'. The 'Sink dataset' is 'ds\_adls\_bronze\_parquet'. The 'Dataset properties' table shows the following configuration:

| Name        | Value                                 | Type   |
|-------------|---------------------------------------|--------|
| folder_path | @concat('bronze/', item().table_name) | string |
| file_name   | data.parquet                          | string |

The 'Copy behavior' is set to 'Select...'. Other properties like 'Max concurrent connections', 'Block size (MB)', 'Metadata', and 'Max rows per file' are also visible. The right sidebar shows the 'Properties' panel with the 'Name' field set to 'pl\_ingest\_bronze'.

*False branch, sink: bronze/<table\_name>/data.parquet, a flat folder per table*

# 11

STEP 11

## Advance the Watermark

### WHAT

After each incremental copy succeeds, a Stored procedure (etl.usp\_update\_watermark) sets that table's last\_loaded\_at to run\_start\_at. It runs only on the copy's success (success dependency).

### WHY

*The watermark is the memory of how far the last run got. Advancing to run\_start\_at, and only on success, means the next run pulls only newer rows and a failed copy leaves the watermark untouched for a safe retry.*

# Advance the Watermark

The screenshot displays the Microsoft Azure Data Factory portal interface. The left sidebar shows the 'Factory Resources' pane with a tree view containing Pipelines, Change Data Capture (preview), and Datasets. The 'pL\_ingest\_bronze' pipeline is selected. The main canvas shows the pipeline design with a 'Copy data' activity (labeled 'Copy incremental') connected to a 'Stored procedure' activity (labeled 'Advance watermark'). The 'Stored procedure' activity is expanded, showing its configuration in the 'Settings' tab. The 'Linked service' is set to 'ls\_azuresql'. The 'Stored procedure name' is '[etl].[usp\_update\_watermark]'. The 'Stored procedure parameters' section shows two parameters: 'run\_start\_at' of type 'DateTime' with value '@variables('run\_start\_at')', and 'table\_name' of type 'String' with value '@item().table\_name'. The 'Properties' pane on the right shows the 'General' tab with the 'Name' set to 'pl\_ingest\_bronze' and a 'Description' field. A 'Cancel' button is visible at the bottom right.

Microsoft Azure | Data Factory | adf-atliq-ab

Factory Resources

- Pipelines: 1
  - pL\_ingest\_bronze
- Change Data Capture (preview): 0
- Datasets: 5
  - ds\_adls\_bronze\_parquet
  - ds\_adls\_csv\_marketing
  - ds\_adls\_csv\_supplier
  - ds\_azuresql\_control
  - ds\_azuresql\_generic
- Data flows: 0
- Power Query: 0

Activities

- store
  - Stored procedure

pl\_ingest\_bronze > ForEach source > If incremental > True activities

Copy data

- Copy incremental

Stored procedure

- Advance watermark

General Settings User properties

Linked service \*  Test connection Edit + New

Stored procedure name \*  Refresh

☐ Enter manually

Stored procedure parameters

Import + New Delete

| Name                                  | Type *   | Value                      |
|---------------------------------------|----------|----------------------------|
| <input type="checkbox"/> run_start_at | DateTime | @variables('run_start_at') |
| <input type="checkbox"/> table_name   | String   | @item().table_name         |

> Advanced

Properties

General Related

Name \*

Description

Annotations

+ New

Cancel

Advance watermark: usp\_update\_watermark with run\_start\_at and table\_name, after Copy incremental

# 12

STEP 12

## Copy the Flat Files

### WHAT

Two dedicated Copy activities on the outer canvas convert `supplier_price_list.csv` and `marketing_spend.csv` to Parquet in Bronze, running in parallel with the SQL loads.

### WHY

*The CSVs are a different source type, so they get their own copies rather than being forced through the metadata loop. Running them in parallel with the SQL loads keeps total ingestion time low.*

## Copy the Flat Files

The screenshot displays the Microsoft Azure Data Factory portal interface. The top navigation bar shows the URL: [https://adf.azure.com/en/authoring/pipeline/pl\\_ingest\\_bronze?factory=%2Fsubscriptions%2Ffe7a04e5-37b1-4992-a127-541400ea45ad%2FresourceGroups%2Frg-atliq-capsto...](https://adf.azure.com/en/authoring/pipeline/pl_ingest_bronze?factory=%2Fsubscriptions%2Ffe7a04e5-37b1-4992-a127-541400ea45ad%2FresourceGroups%2Frg-atliq-capsto...). The left sidebar shows the 'Factory Resources' pane with a search bar and a list of resources including Pipelines (1), Change Data Capture (preview) (0), and Datasets (5). The 'Pipelines' section is expanded, showing the 'pl\_ingest\_bronze' pipeline. The main canvas displays the pipeline design, which includes a 'Set variable' activity (Set run\_start\_at), a 'Lookup' activity (Lookup control\_table), a 'ForEach' activity (ForEach source), and two 'Copy data' activities (Copy supplier CSV and Copy marketing CSV). The 'Properties' pane on the right shows the 'General' tab for the 'pl\_ingest\_bronze' pipeline, with fields for Name, Description, and Annotations. The 'Pipeline validation output' pane on the right shows a success message: 'Your pipeline has been validated. No errors were found.' The bottom of the canvas has tabs for Parameters, Variables, Settings, and Output, with a '+ New' button under Parameters.

*pl\_ingest\_bronze canvas, validated: Set variable, Lookup, ForEach and the two CSV copies*

## Copy the Flat Files

The screenshot displays the Microsoft Azure Data Factory portal interface. The top navigation bar shows the URL and user information. The left sidebar contains the 'Factory Resources' pane with a search bar and a list of resources including Pipelines, Change Data Capture (preview), and Datasets. The 'pl\_ingest\_bronze' pipeline is selected. The main canvas shows the pipeline flow: 'Set variable' (Set run\_start\_at) -> 'Lookup' (Lookup control table) -> 'ForEach' (ForEach source) -> 'Copy data' (Copy supplier CSV). The 'Copy data' activity is selected, and its 'Source' tab is active. The 'Source dataset' is 'ds\_adls\_csv\_supplier'. The 'File path type' is 'File path in dataset'. The 'Start time (UTC)' and 'End time (UTC)' are empty. 'Filter by last modified' is checked. 'Recursively' is checked. 'Enable partitions discovery' is unchecked. 'Max concurrent connections' is empty. 'Skip line count' is empty. 'Additional columns' is empty. The 'Properties' pane on the right shows the 'General' tab with 'Name' 'pl\_ingest\_bronze' and 'Description' empty. The 'Annotations' section has a '+ New' button. The 'Cancel' button is at the bottom right.

Copy supplier CSV source: `ds_adls_csv_supplier` from the landing folder

## Copy the Flat Files

The screenshot displays the Microsoft Azure Data Factory (ADF) console. The left sidebar shows the 'Factory Resources' tree with 'Pipelines' expanded, highlighting the 'pl\_ingest\_bronze' pipeline. The main canvas shows the pipeline's logic flow: 'Set variable' (Set run\_start\_at) → 'Lookup' (Lookup control table) → 'ForEach' (ForEach source) → 'Copy data' (Copy supplier CSV). The 'Copy data' activity is selected, and its configuration pane is open, showing the 'Sink' tab. The sink is configured with the dataset 'ds\_adls\_bronze\_parquet'. The 'Dataset properties' table is as follows:

| Name        | Value                      | Type   |
|-------------|----------------------------|--------|
| folder_path | bronze/supplier_price_list | string |
| file_name   | data.parquet               | string |

Below the table, the 'Copy behavior' is set to 'Select...'. Other settings like 'Max concurrent connections', 'Block size (MB)', 'Metadata', and 'Max rows per file' are visible but not configured. The right sidebar shows the 'Properties' pane for the selected activity, with the 'Name' field set to 'pl\_ingest\_bronze'.

Copy supplier CSV sink: bronze/supplier\_price\_list/data.parquet

# 13

STEP 13

## Run and Verify Bronze

### WHAT

Validated, published, and Debug-ran the pipeline. All activities succeeded, and the lakehouse now holds a bronze/ tree: flat folders for full loads, ingest\_date= partitions for incremental loads.

### WHY

*A successful run proves the wiring, but the deliverable is data in Bronze. The visible folder shape (flat vs date-partitioned) confirms the full and incremental strategies behaved exactly as designed.*

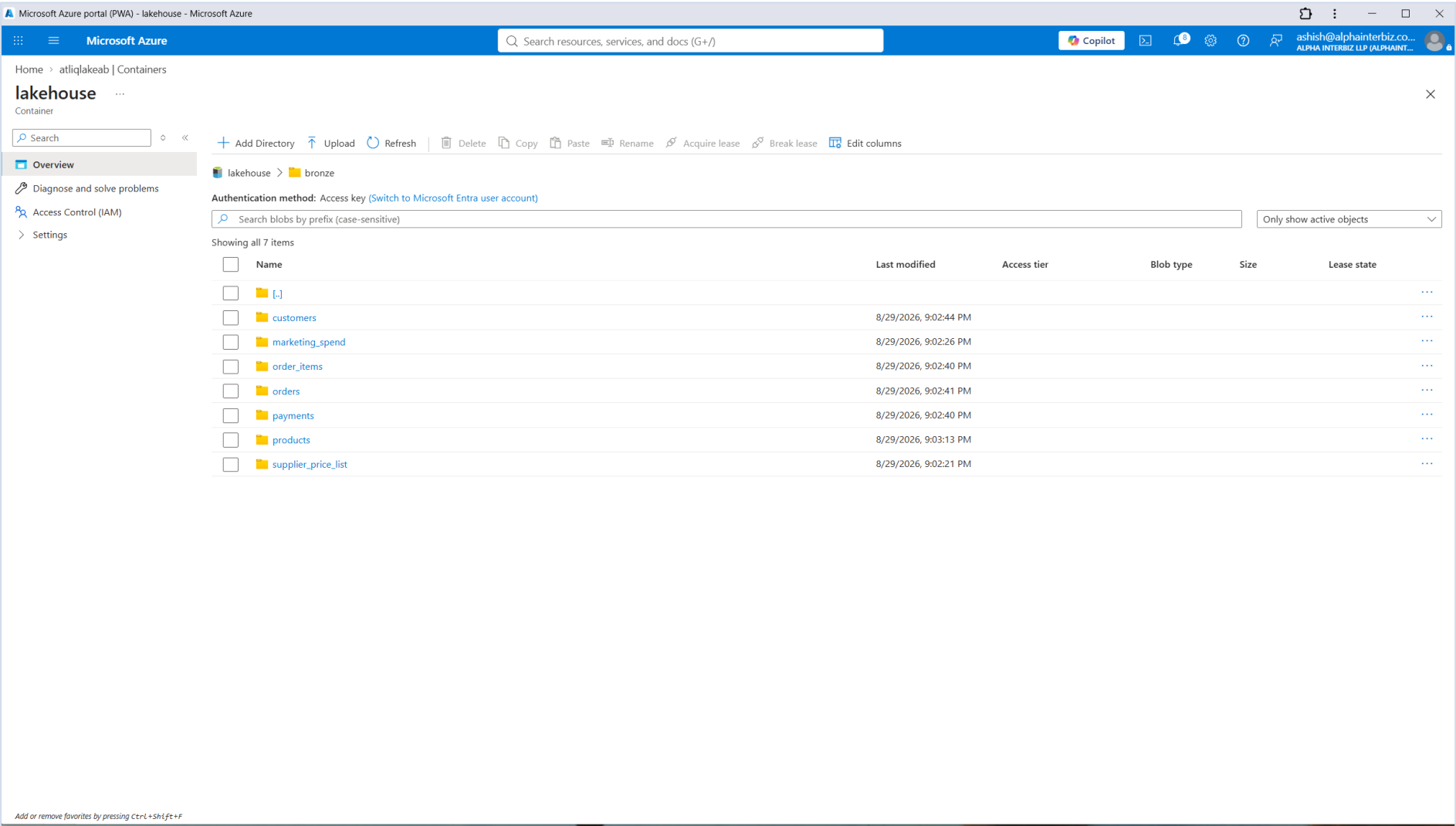
## Run and Verify Bronze

The screenshot displays the Microsoft Azure Data Factory portal interface. The top navigation bar shows the URL: [https://adf.azure.com/en/authoring/pipeline/pl\\_ingest\\_bronze?factory=%2Fsubscriptions%2Ffe7a04e5-37b1-4992-a127-541400ea45ad%2FresourceGroups%2Frg-atliq-capsto...](https://adf.azure.com/en/authoring/pipeline/pl_ingest_bronze?factory=%2Fsubscriptions%2Ffe7a04e5-37b1-4992-a127-541400ea45ad%2FresourceGroups%2Frg-atliq-capsto...). The left sidebar shows the 'Factory Resources' section with a list of pipelines, including 'pl\_ingest\_bronze'. The main canvas displays the 'pl\_ingest\_bronze' pipeline diagram, which includes activities like 'Set variable', 'Lookup', 'ForEach', 'Copy data', and 'Copy data'. The 'Output' tab is selected, showing a table of activity results. The table indicates that all activities were successful.

| Activity name     | Activity st... | Activit...     | Run start             | Duration | Integration runtime                           |
|-------------------|----------------|----------------|-----------------------|----------|-----------------------------------------------|
| Copy full         | ✓ Succeeded    | Copy data      | 8/29/2026, 9:02:52 PM | 25s      | AutoResolveIntegrationRuntime (Central India) |
| If incremental    | ✓ Succeeded    | If Condition   | 8/29/2026, 9:02:51 PM | 28s      |                                               |
| Advance watermark | ✓ Succeeded    | Stored procedu | 8/29/2026, 9:02:45 PM | 14s      | AutoResolveIntegrationRuntime (Central India) |
| Advance watermark | ✓ Succeeded    | Stored procedu | 8/29/2026, 9:02:45 PM | 14s      | AutoResolveIntegrationRuntime (Central India) |
| Advance watermark | ✓ Succeeded    | Stored procedu | 8/29/2026, 9:02:44 PM | 15s      | AutoResolveIntegrationRuntime (Central India) |
| Copy full         | ✓ Succeeded    | Copy data      | 8/29/2026, 9:02:26 PM | 23s      | AutoResolveIntegrationRuntime (Central India) |
| Copy incremental  | ✓ Succeeded    | Copy data      | 8/29/2026, 9:02:25 PM | 20s      | AutoResolveIntegrationRuntime (Central India) |
| Copy incremental  | ✓ Succeeded    | Copy data      | 8/29/2026, 9:02:25 PM | 19s      | AutoResolveIntegrationRuntime (Central India) |
| Copy incremental  | ✓ Succeeded    | Copy data      | 8/29/2026, 9:02:25 PM | 19s      | AutoResolveIntegrationRuntime (Central India) |

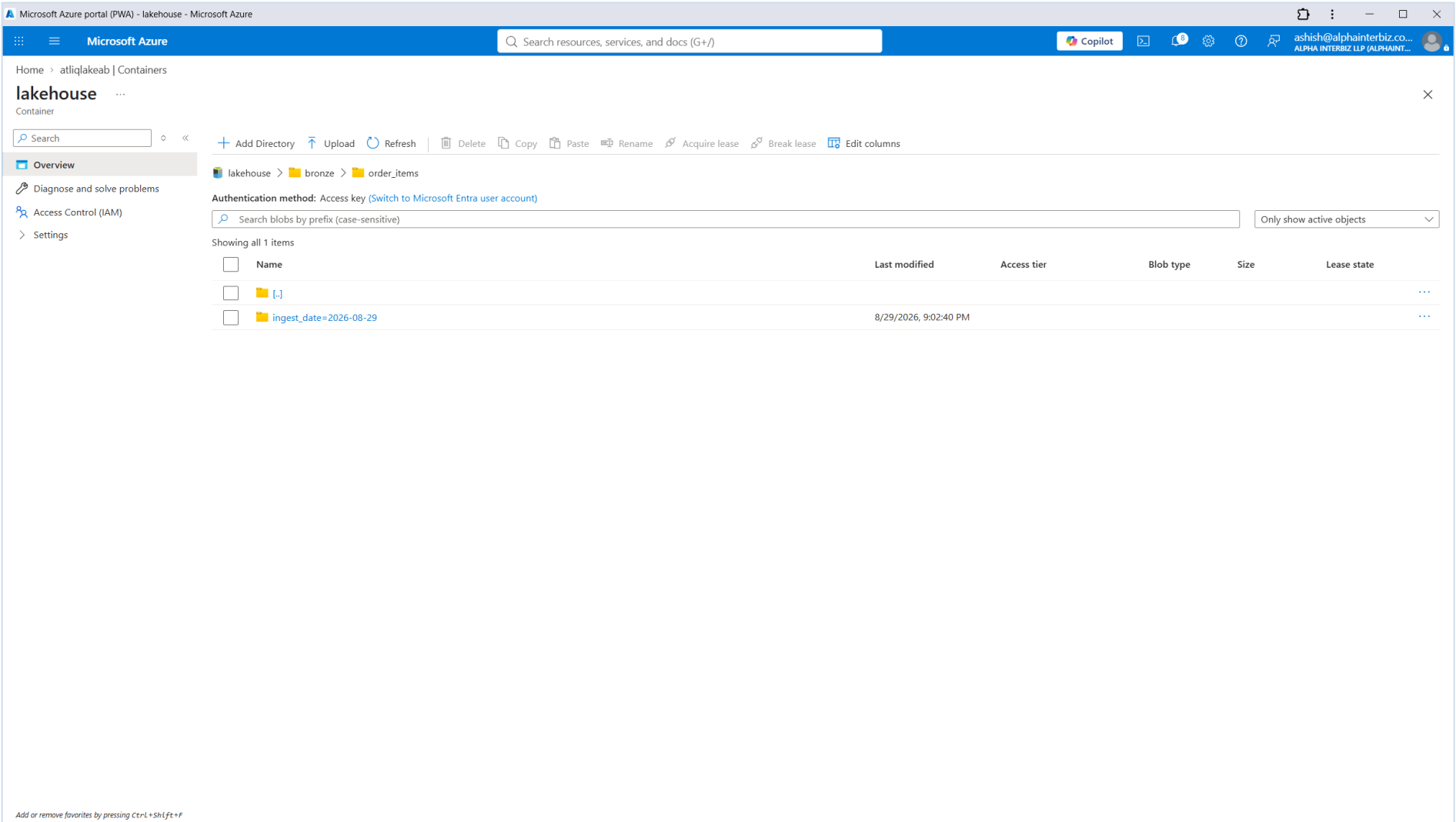
Debug run: pipeline status Succeeded, every activity green

# Run and Verify Bronze



lakehouse/bronze/ after the run: one folder per source, 7 in total

# Run and Verify Bronze



bronze/order\_items/ written as an ingest\_date= partition

# 14

STEP 14

## Prove Idempotency

### WHAT

Ran the pipeline a second time. The incremental tables copied 0 rows (Rows read: 0, Rows written: 0), because their watermarks had advanced past all existing data on the first run.

### WHY

*Anyone can build a pipeline that runs once. This proves it behaves correctly every night: it moves only new data, never double-counts, and is safe to re-run or retry. This is the core reliability property of the design.*

## Prove Idempotency

The screenshot displays the Microsoft Azure Data Factory portal interface. On the left, the 'Factory Resources' pane shows a list of pipelines, datasets, and data flows. The 'pl\_ingest\_bronze' pipeline is selected. The main area shows the details of the 'pl\_ingest\_bronze' activity, which is a 'Copy' activity. The activity is shown as 'Succeeded' with a green checkmark. The details include:

- Activity run id:** 50e9b83d-b238-4da8-b2ef-8dc88087b407
- Data read:** 0 byte
- Rows read:** 0
- Peak connections:** 1
- Data written:** 309 bytes
- Files written:** 1
- Rows written:** 0
- Peak connections:** 1
- Copy duration:** 00:00:14
- Throughput:** 0 byte/s
- Start time:** 8/29/2026, 9:11:30 PM
- Used DIUs:** 4
- Used parallel copies:** 1
- Duration:** 00:00:14
- Queue:** 00:00:07
- Transfer:** 00:00:05
- Data consistency verification:** Not verified

The 'Properties' pane on the right shows the 'General' tab with the 'Name' field set to 'pl\_ingest\_bronze' and a 'Description' field. The 'Annotations' section has a '+ New' button.

At the bottom of the screen, a status bar shows the activity 'Copy marketing CSV' with a green checkmark and the text 'Succeeded'. Below this, it shows 'Copy data' with a green checkmark and the text 'Succeeded'. The status bar also displays the date and time '8/29/2026, 9:11:10 PM' and the duration '21s'. The status bar also displays the text 'AutoResolveIntegrationRuntime (Central India)'.

Second run, copy details: Rows read 0, Rows written 0

# Milestone 2 Complete

What this milestone delivered

## Metadata-driven pipeline

One generic pipeline reads etl.control\_table and loads every source. New sources need only a new row, not a new pipeline.

## Bronze lake populated

All 5 SQL tables plus 2 CSVs landed as Parquet. Full loads flat, incremental loads partitioned by ingest date.

## Proven idempotent

A second run copied 0 incremental rows. The nightly sync moves only new data and is safe to re-run.

## Key design ideas demonstrated

**Metadata-driven ingestion** (control table)   **Watermark incremental loads** (only the delta)   **Secrets in Key Vault** (no inline password)

**Date-partitioned Bronze** (audit trail)   **Success-gated watermark advance** (safe retry)   **Idempotent re-runs**

*Cost to date: negligible (storage + a few pipeline runs). No Databricks compute used yet.*